

# Ariel Szekely

---

## Contact Information

Phone +1 (817) 876-2431  
Email arielck@mit.edu  
Homepage www.arielck.com

---

## Education

- 2020-present **Ph.D. in Computer Science**, *Massachusetts Institute of Technology*, Cambridge, MA
- 2020-2022 **M.S. in Computer Science**, *Massachusetts Institute of Technology*, Cambridge, MA,  
Thesis:  *$\sigma$ OS: Elastic Realms for Multi-Tenant Cloud Computing*  
GPA: 5.0/5.0
- 2016-2020 **B.S. in Computer Science (Honors)**, *The University of Texas at Austin*, Austin, TX,  
Thesis: *Exploring GPU Timing Side-Channels*  
GPA: 3.94/4.0

## Interests

Distributed Systems, Operating Systems, Security, Concurrency.

---

## Publications

- [1] **Ariel Szekely**, Adam Belay, Robert Morris, and M. Frans Kaashoek. Unifying serverless and microservice workloads with sigmaos. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles, SOSP '24*, page 385–402, New York, NY, USA, 2024. Association for Computing Machinery.
- [2] Henrique Fingler, Isha Tarte, Hangchen Yu, **Ariel Szekely**, Bodun Hu, Aditya Akella, and Christopher J. Rossbach. Towards a machine learning-assisted kernel with lake. In *18th Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 23)*, Vancouver, Canada, March 2023. Association for Computing Machinery.
- [3] Tyler Hunt, Zhipeng Jia, Vance Miller, **Ariel Szekely**, Yige Hu, Christopher J. Rossbach, and Emmett Witchel. Telekine: Secure computing with cloud gpus. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, Santa Clara, CA, February 2020. USENIX Association.

---

## Awards

- 2022 NSF Graduate Research Fellowship
- 2020 MIT Lemelson Presidential Fellowship
- 2020 MIT EECS Departmental Fellowship
- 2020 Dean's Honored Graduate, UT Austin College of Natural Sciences
- 2020 Research Distinction, UT Austin College of Natural Sciences
- 2020 NSF Graduate Research Fellowship Honorable Mention
- 2020 CRA Outstanding Undergraduate Researcher Honorable Mention
- 2019 NSF REU
- 2019 UT Endowed Presidential Scholarship

- 2018,19,20 UT College of Natural Sciences College Scholar  
2018 Louis E. Rosier Memorial Scholarship  
2016 UT CNS Freshman Scholarship  
2016 National Merit Scholarship

## Professional Experience

- 2023 **Student Researcher**, *Google Systems Research Group*, Seattle, WA  
2019 **Research Assistant**, *The University of Texas at Austin*, Austin, TX  
2018 **Software Engineer Intern**, *Bloomberg L.P.*, New York City, NY  
2017 **Software Engineer Intern**, *AT&T*, Dallas, TX

## Teaching Experience

- 2023 **TA: Computer Systems Security**, *Massachusetts Institute of Technology*  
Instructors: Nickolai Zeldovich  
2022 **TA: Operating Systems Engineering**, *Massachusetts Institute of Technology*  
Instructors: Frans Kaashoek, Robert Morris  
2020 **TA: Concurrency (Honors)**, *The University of Texas at Austin*  
Instructor: Christopher J. Rossbach  
2020 **TA: Intro to CS Research (Honors)**, *The University of Texas at Austin*  
Instructor: Calvin Lin  
2019 **TA: Intro to CS Research (Honors)**, *The University of Texas at Austin*  
Instructor: Calvin Lin

## Service

- 2022-present **EECS Communication Lab Fellow**, *Massachusetts Institute of Technology*, Cambridge, MA  
2021 **SOSP '21 Artifact Evaluation Committee**, *SOSP 2021*, Kolbenz, Germany  
2020-present **MIT Graduate Application Assistance Program Mentor**, *Massachusetts Institute of Technology*, Cambridge, MA  
2020 **MIT AI Undergraduate Mentorship Program Mentor**, *Massachusetts Institute of Technology*, Cambridge, MA  
2017-2020 **UTCS Ambassador**, *The University of Texas at Austin*, Austin, TX

## Languages

- English Fluent  
Spanish Fluent  
French Intermediate

## **Research Statement**

Traditionally, performance and security have been seen as opposing goals when designing cloud platforms. Isolating different tenants' cloud applications requires deploying heavyweight isolation technology, like virtual machines (VMs) which are slow to start and impose significant performance overhead. Achieving good performance and fast start times for applications requires forgoing strong isolation. The thesis of my work is that this is a false dichotomy. By carefully redesigning the interfaces available to tenants' applications, cloud platforms can provide good performance without sacrificing security.

My first PhD project, SigmaOS, is one example of this philosophy. SigmaOS is a novel, multi-tenant cloud operating system (OS) which achieves good performance and strong security with a *cloud-centric* API. The key observation behind SigmaOS is that modern cloud applications have come to rely on cloud services for much of the work traditionally done by the local OS, and can be supported with a much narrower OS interface which is cheap to isolate.

For example, cloud applications use services like Amazon S3 for long-term storage instead of the local file system and spawn additional computations via a cloud scheduler instead of forking additional local processes. Rather than provide a full Linux environment to run each application, SigmaOS omits many OS services which are expensive to isolate. SigmaOS doesn't provide applications with writable local storage, a per-application Linux image, or dedicated IP addresses. Instead, SigmaOS runs applications in a minimal, restrictive container which provides just the facilities needed to implement most cloud applications, *and no more*. In particular, SigmaOS allows a small set of syscalls for memory management, threads, timers, signals, and a custom secure networking API similar to many high-level programming languages' networking packages.

SigmaOS is able to cold-start its secure, lightweight containers in 7.7 milliseconds, and warm-start containers in 2 milliseconds. SigmaOS's warm-starts are 23x faster than AWS Lambda (the leading serverless function platform) and 105x faster than Kubernetes containers (the leading container orchestration platform). SigmaOS is competitive with state-of-the-art research systems which start containers in single-digit milliseconds, but unlike these systems SigmaOS does not sacrifice security guarantees in order to achieve its performance. We built several applications on top of SigmaOS, including a MapReduce framework, several microservice applications, an image processing application, and a fault-tolerant key-value cache. **SigmaOS was published as the Symposium on Operating Systems Principles (SOSP) 2024**, the leading venue in my field, and is an active research project. We have had 4 students write Master's theses on extensions of SigmaOS, and currently have another 3 Master's students planning on completing their theses in the coming Spring semester.

My current research project proposes that developers structure cloud applications as two sandboxes, a *light* sandbox and a *heavy* sandbox. This arrangement creates opportunities for several interesting optimizations for the cloud platform. First, each sandbox can use a different isolation technology with different tradeoffs. The heavy sandbox can use heavyweight isolation which is slow to start but provides strong security guarantees and supports a wide array of

applications (e.g., VMs). The light sandbox can use lightweight isolation which starts quickly, but runs applications in a constrained environment (e.g., WebAssembly language-level isolation). Since the light sandbox starts quickly, it can overlap with the heavy sandbox's startup routine and do much of the application's initialization work, such as fetching state from other services, while the heavy sandbox boots. When the heavy sandbox is up and running, the light sandbox can hand off the initialization results to the heavy sandbox for it to use. This overlap has the potential to cut application start times in half. Faster start times allow tenants to scale up their cloud applications faster in response to changes in load, which translates to lower application deployment costs.

Splitting an application into two sandboxes also allows for closer collaboration between applications and the cloud platform. For example, current cloud platforms either support migration of application instances from one machine to another without application involvement, or push the responsibility of handling migration entirely into the application. Application-involved migration can be more efficient, as the developer can optimize the migration procedure with knowledge of the application. However, making migration fully the responsibility of the application misses opportunities for the platform to participate and optimize migration using its knowledge of the full cluster state.

Using two sandboxes allows the platform and application to collaborate around cloud management tasks like migration. The application can specify *how* migration should happen in a light sandbox, and the platform can decide *when* and *where* migration happens by running the light sandbox on the target machine at a time of its choosing. This allows application and platform to share responsibility for cloud management tasks, and allows the application to safely customize platform behavior to optimize its performance.

Finally, deploying applications as two sandboxes enables developers to set different resource reservations for each sandbox. Many applications have I/O-heavy initialization phases, in which they read in the state they need in order to operate, and then transition to more compute-heavy execution phases when handling user requests. Splitting the initialization and execution phases into their own sandboxes allows the application to express the different resource reservations required for each phase to the platform. This is beneficial for both applications and platforms: applications pay for only the resources they need in each phase, and platforms use the knowledge of applications' shifting resource demands to make better cluster-level scheduling decisions and better utilize their cloud resources. This work is ongoing, and is targeted for **submission to Operating Systems Design and Implementation (OSDI) in December 2025.**

In my last two years as a PhD student at MIT, I plan to continue working towards designing new interfaces to program cloud applications efficiently, securely, and with good performance. I believe doing so is crucially important in order to manage growing demand for datacenter compute resources, and to use these resources to their maximum potential.